

GitHub link inside my Colab notebook

```
from IPython.display import display, Markdown

github_link = "https://github.com/bharath18092006/student-performance-xai"

display(Markdown(f"""
# GitHub Repository

[Click here to view the complete project on GitHub]({github_link})
"""))
```

GitHub Repository

[Click here to view the complete project on GitHub](https://github.com/bharath18092006/student-performance-xai)

1. Install libraries

```
!pip install -q shap lime
```

```
275.7/275.7 kB 5.7 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Building wheel for lime (setup.py) ... done
```

2. Import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import shap

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from lime.lime_tabular import LimeTabularExplainer
```

3. Download Student Performance dataset

```
!wget -q -O student.zip "https://archive.ics.uci.edu/ml/machine-learning-databases/00320/student.zip"

!unzip -o -q student.zip

df = pd.read_csv("student-mat.csv", sep=";")

print("Dataset Shape:", df.shape)

display(df.head())
```

Dataset Shape: (395, 33)

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3	
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	4	1	1	3	6	5	6	6	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	3	1	1	3	4	5	5	6	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	2	2	3	3	10	7	8	10	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	2	1	1	5	2	15	14	15	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	2	1	2	5	4	6	10	10	

5 rows × 33 columns

4. Understand the dataset

```
print("Columns:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())
```

```
26 Dalc      395 non-null    int64
27 Walc      395 non-null    int64
28 health    395 non-null    int64
29 absences  395 non-null    int64
30 G1        395 non-null    int64
31 G2        395 non-null    int64
32 G3        395 non-null    int64
```

```
dtypes: int64(16), object(17)
```

```
memory usage: 102.0+ KB
```

```
Missing Values:
```

```
school      0
sex         0
age         0
address     0
famsize     0
Pstatus     0
Medu        0
Fedu        0
Mjob        0
Fjob        0
reason      0
guardian    0
traveltime  0
studytime   0
failures    0
schoolsup   0
famsup      0
paid        0
activities  0
nursery     0
higher      0
internet    0
romantic    0
famrel      0
freetime    0
goout       0
Dalc        0
Walc        0
health      0
absences    0
G1          0
G2          0
G3          0
```

```
dtype: int64
```

5. Select input and target

```
target = "G3"

X = df.drop(columns=[target])

y = df[target]

print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

```
Input shape: (395, 32)
Target shape: (395,)
```

6. Convert categorical data into numerical data

```
X = pd.get_dummies(X, drop_first=True)

print("After encoding:")
print("Number of features:", X.shape[1])

display(X.head())
```

```
After encoding:
Number of features: 41
```

	age	Medu	Fedu	traveltime	studytime	failures	famrel	freetime	goout	Dalc	...	guardian_mother	guardian_other	schoolsup_yes	famsup_yes	paid_y
0	18	4	4	2	2	0	4	3	4	1	...	True	False	True	False	Fal
1	17	1	1	1	2	0	5	3	3	1	...	False	False	False	True	Fal
2	15	1	1	1	2	3	4	3	2	2	...	True	False	True	False	Tr
3	15	4	2	1	3	0	3	2	2	1	...	True	False	False	True	Tr
4	16	3	3	1	2	0	4	3	2	1	...	False	False	False	True	Tr

```
5 rows × 41 columns
```

7. Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print("Training samples:", X_train.shape[0])  
print("Testing samples :", X_test.shape[0])
```

```
Training samples: 316  
Testing samples : 79
```

8. Create and train ML model

```
model = RandomForestRegressor(  
    n_estimators=300,  
    max_depth=12,  
    random_state=42,  
    n_jobs=-1  
)  
  
model.fit(X_train, y_train)  
  
print("Model training completed!")
```

```
Model training completed!
```

9. Evaluate the model

```
y_pred = model.predict(X_test)  
  
mae = mean_absolute_error(y_test, y_pred)  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2 = r2_score(y_test, y_pred)  
  
print("==== MODEL PERFORMANCE ====")  
print(f"MAE : {mae:.4f}")  
print(f"RMSE : {rmse:.4f}")  
print(f"R² : {r2:.4f}")  
print(f"R² % : {r2 * 100:.2f}%")
```

```
==== MODEL PERFORMANCE ====  
MAE : 1.1694  
RMSE : 1.9734  
R² : 0.8101  
R² % : 81.01%
```

10. Predict a student's performance

```
student_index = 0

student = X_test.iloc[[student_index]]

prediction = model.predict(student)[0]

actual = y_test.iloc[student_index]

print("==== STUDENT PERFORMANCE PREDICTION ====")
print("Predicted Final Grade:", round(prediction, 2))
print("Actual Final Grade   :", actual)
```

```
==== STUDENT PERFORMANCE PREDICTION ====
Predicted Final Grade: 8.27
Actual Final Grade   : 10
```

11. Display student's input features

```
print("Student Input Features:")
display(student.T)
```

Student Input Features:

1 to 25 of 41 entries

Filter



index	
age	17
Medu	2
Fedu	1
traveltime	2
studytime	1
failures	3
famrel	4
freetime	5
goout	1
Dalc	1
Walc	1
health	3
absences	2
G1	8
G2	8
school_MS	false
sex_M	true
address_U	true
famsize_LE3	false
Pstatus_T	true
Mjob_health	false
Mjob_other	true
Mjob_services	false
Mjob_teacher	false
Fjob_health	false

Show 25 per page

1

2

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables

12. Implement SHAP

```
shap_explainer = shap.TreeExplainer(model)

shap_values_student = shap_explainer.shap_values(student)

print("SHAP explanation generated successfully!")

SHAP explanation generated successfully!
```

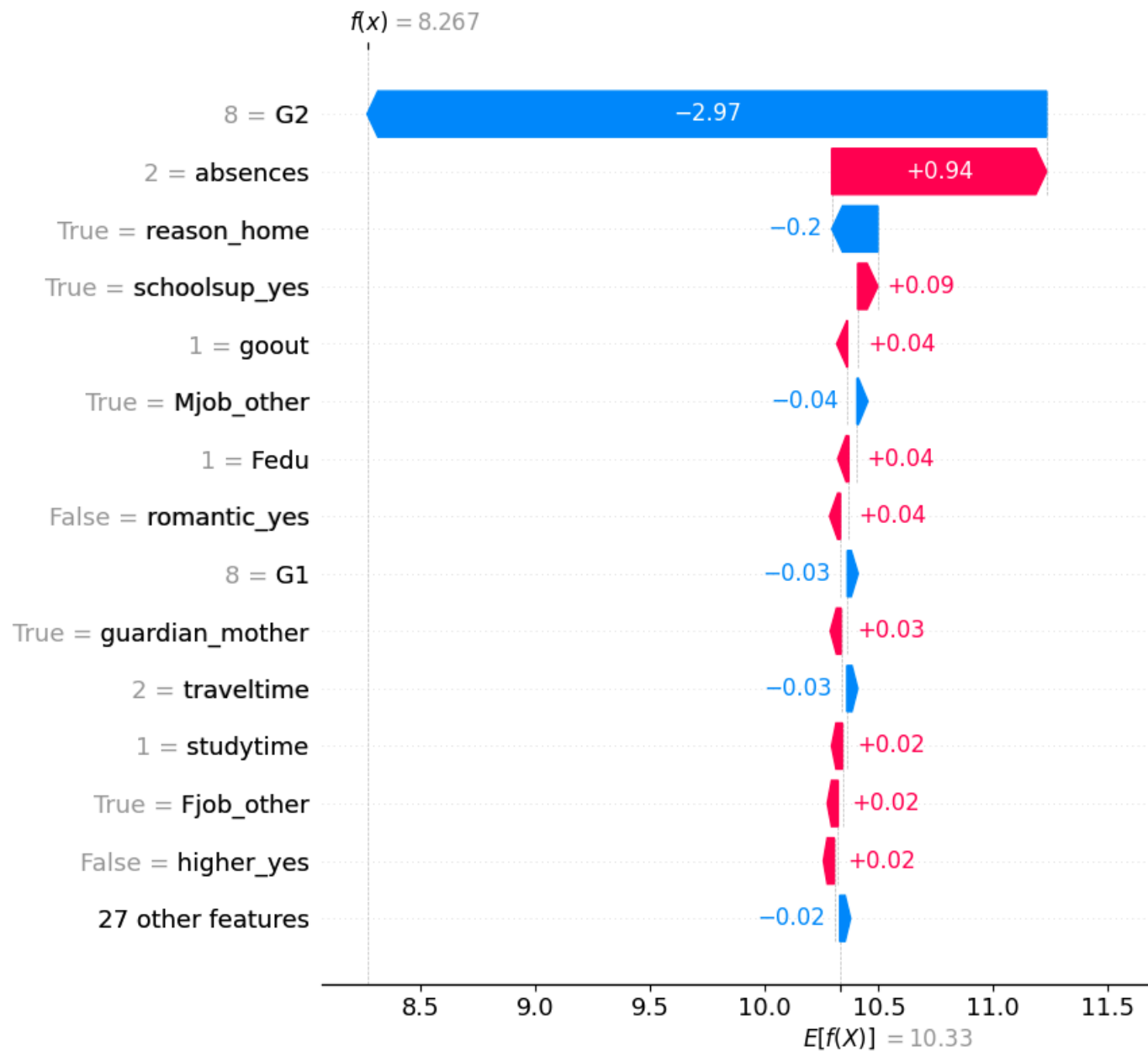
13. SHAP Waterfall Plot

```
base_value = shap_explainer.expected_value

if isinstance(base_value, np.ndarray):
    base_value = base_value[0]

shap_explanation = shap.Explanation(
    values=shap_values_student[0],
    base_values=base_value,
    data=student.iloc[0].values,
    feature_names=student.columns.tolist()
)

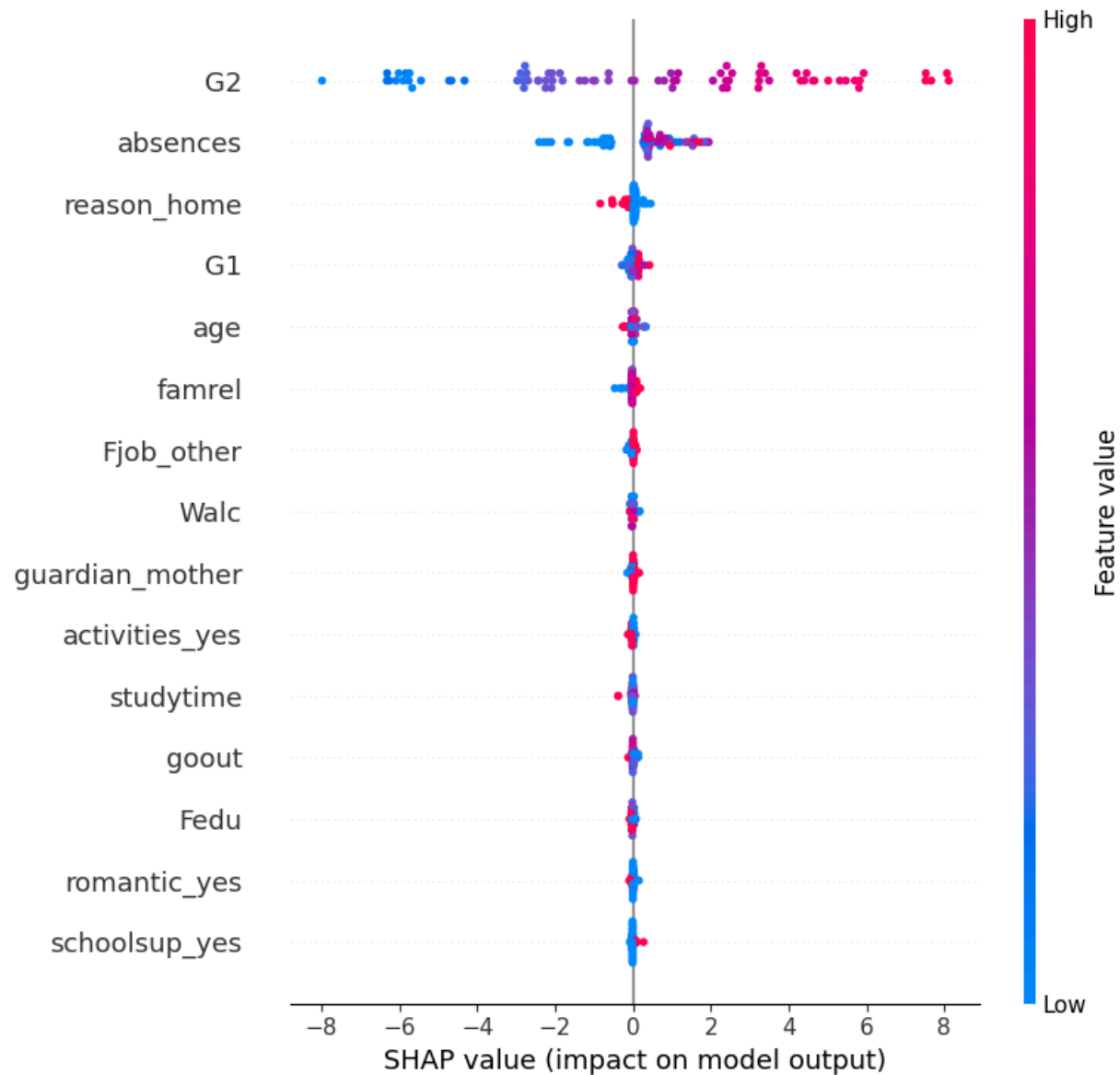
shap.plots.waterfall(
    shap_explanation,
    max_display=15
)
```



14. SHAP Global Feature Importance

```
shap_values_test = shap_explainer.shap_values(X_test)

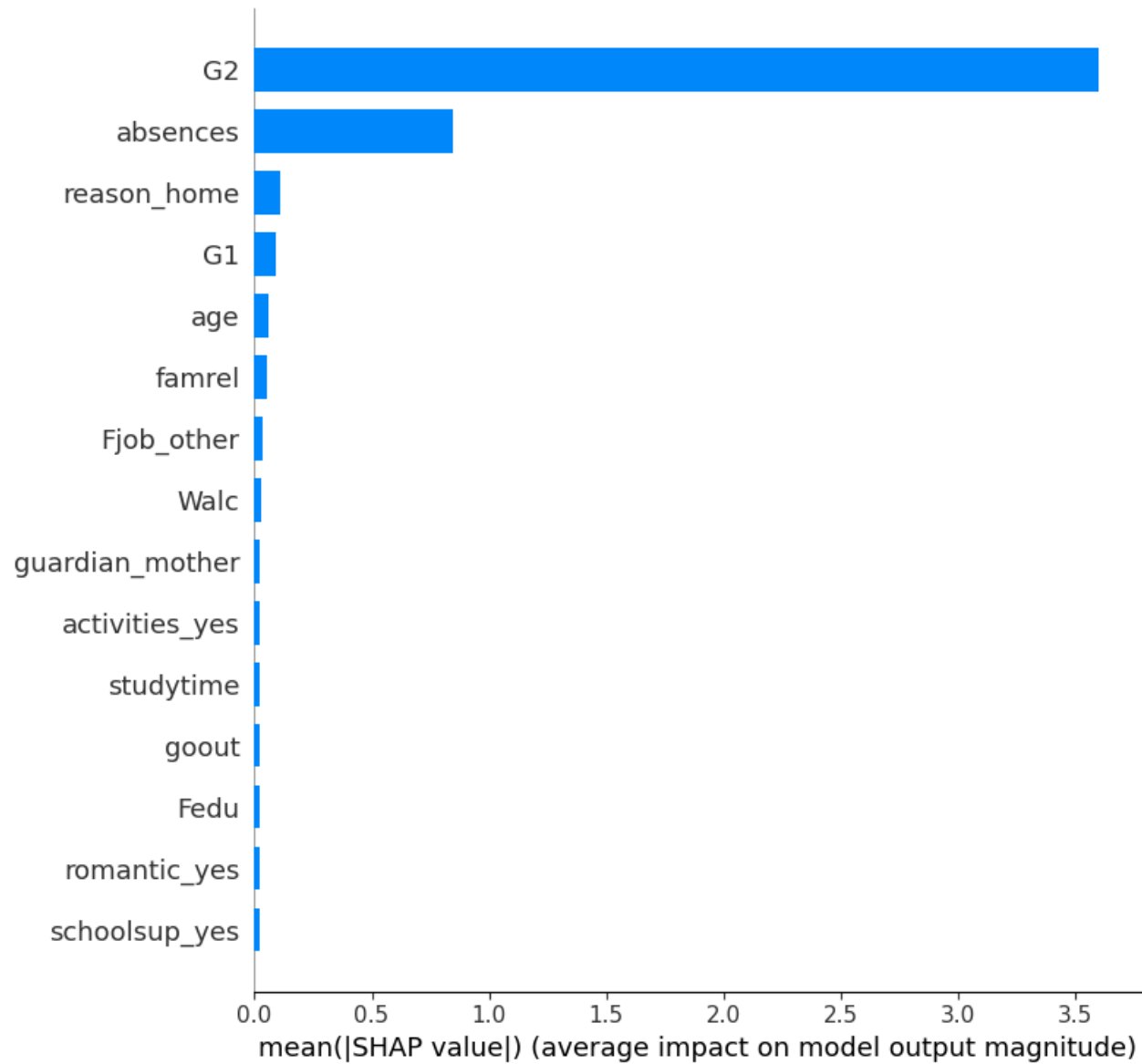
shap.summary_plot(
    shap_values_test,
    X_test,
    max_display=15
)
```



15. SHAP Bar Plot

```
shap.summary_plot(  
    shap_values_test,  
    X_test,
```

```
plot_type="bar",  
max_display=15  
)
```



16. Create SHAP feature table

```
shap_df = pd.DataFrame({
    "Feature": X_test.columns,
    "SHAP_Value": shap_values_student[0]
})

shap_df["Absolute_SHAP"] = shap_df["SHAP_Value"].abs()

shap_df = shap_df.sort_values(
    "Absolute_SHAP",
    ascending=False
)

print("==== TOP SHAP FEATURES ====")

display(
    shap_df.head(15)[
        ["Feature", "SHAP_Value"]
    ]
)
```

===== TOP SHAP FEATURES =====

	Feature	SHAP_Value	
14	G2	-2.966030	
12	absences	0.937483	
28	reason_home	-0.200743	
33	schoolsup_yes	0.087479	
8	goout	0.044759	
21	Mjob_other	-0.041613	
2	Fedu	0.036915	
40	romantic_yes	0.035913	
13	G1	-0.030593	
31	guardian_mother	0.027468	
3	traveltime	-0.025534	
4	studytime	0.019824	
25	Fjob_other	0.018615	
38	higher_yes	0.017044	
0	age	-0.015439	

17. Implement LIME

```
lime_explainer = LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=X_train.columns.tolist(),
    mode="regression",
    discretize_continuous=True,
    random_state=42
)

print("LIME explainer created successfully!")

LIME explainer created successfully!
```

18. Generate LIME explanation

```
lime_explanation = lime_explainer.explain_instance(
    student.iloc[0].values,
    model.predict,
    num_features=15
)
```

```
print("==== LIME EXPLANATION ====")
```

```
for feature, weight in lime_explanation.as_list():
    print(f"{feature:50s} : {weight:.4f}")
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestRegressor was
warnings.warn(
==== LIME EXPLANATION =====
G2 <= 9.00 : -7.1349
0.00 < absences <= 4.00 : 0.7009
school_MS <= 0.00 : 0.2530
0.00 < reason_home <= 1.00 : -0.1690
G1 <= 8.00 : -0.1651
schoolsup_yes > 0.00 : 0.1417
reason_other <= 0.00 : -0.1377
reason_reputation <= 0.00 : -0.1119
Walc <= 1.00 : 0.1076
Dalc <= 1.00 : -0.1053
Fjob_services <= 0.00 : 0.1030
Fedu <= 2.00 : 0.0937
guardian_other <= 0.00 : -0.0919
16.00 < age <= 17.00 : 0.0910
Mjob_services <= 0.00 : -0.0865
```

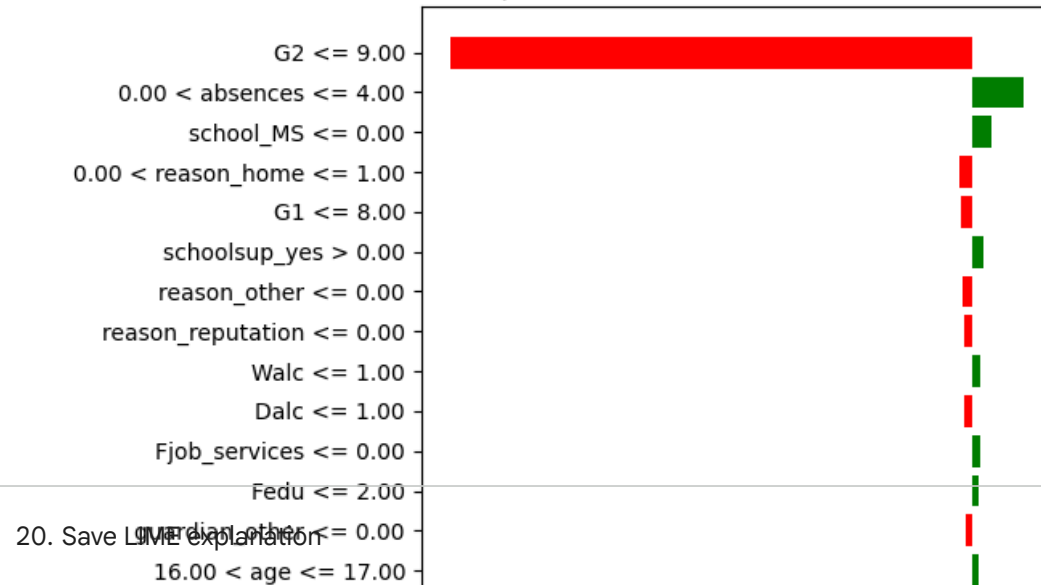
19. Display LIME graph

```
fig = lime_explanation.as_pyplot_figure()

plt.title("LIME Explanation - Student Performance Prediction")

plt.tight_layout()
plt.show()
```

LIME Explanation - Student Performance Prediction



20. Save LIME explanation

```
lime_explanation.save_to_file(
    "lime_student_explanation.html"
)

print("LIME explanation saved as HTML!")
```

LIME explanation saved as HTML!

21. Compare SHAP and LIME

```
print("=====")
print("      SHAP vs LIME EXPLANATION")
print("=====")

print("\nSHAP:")
print("SHAP explains the contribution of each")
print("feature to the model prediction.")
```